CS 0007: Introduction to Java Lecture 11

Nathan Ong
University of Pittsburgh
October 13, 2016

Annoucements

- October 17 – Fall Break
- October 18 – Monday Schedule
  - Those who are signed up for Monday Lab must attend
  - Those who are signed up for Tuesday Lab are not required to attend, but must complete the lab
  - No lecture

Annoucements

- Project 0 is updated, please download the new version
- Rubric will be released very soon
- This project must be done by yourself

Annoucements

- Project 1 will be released while Project 0 is going on just to get you to start thinking about it, but the due date will be far into the future
- There will only be two projects

ARRAYS, LOOPS, AND RECURSION

A Sudden Need

- I need a function that returns the first 100 terms of the harmonic series.
- Harmonic series:

$$\sum_{n=1}^{\infty} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \dots$$

The Dumb Way

```
public static void main(String[] args)
{
  double term0 = 1.0;
  double term1 = 1.0/2.0;
  double term2 = 1.0/3.0;
  …
}
```

Arrays

- Arrays are ordered lists of things depending on the type you specify
- How do you make one?

```
Type[] name = new Type[size];
```

```
int[] empty = new int[10];
```

```
int[] list = {1,2,3,4};
```

Now What?

- I want the first element
- `list[0]`
- I want the last element
- `list[3]`
- I want the length
- `list.length == 4`
- All operations for `int` are allowed for an individual element
- list[0] + list[2] == list[3]

How do I get every element so I can print it?

- Reference every element
- Many if statements!!!!!! YAAAAAAAAYYYY
- No please don't
- But what else can we do?
- WILD KEYWORD APPEARS

while(){…}

- While a condition is true, run the block
- While not at end of the list, print out the next element

```
while(not at end of list)
{
        System.out.println(next element);
}//end loop while(not at end of list)
```

not at end of list

- How do we know we're at the end of the list?
- `list.length`
- How do we know which one we're at?
  - We need to keep track

```
int currElement = 0;
```

  - starts at 0
  - not equal to the end!

```
currElement != list.length
```

while(){…}

- While not at the end of the list, print out the next element

```
int currElement = 0;
while(currElement != list.length)
{
 System.out.println(<next element>);
}//end loop while(currElement!=list.length)
```

next element

- Do we really want to print out the next element?
- If we start at 0, and the list starts at 0, do we?
- No, we want it to print out the current element and then move to the next one
- How do we print out the current element?

```
System.out.println(list[currElement]);
```

- How do we move to the next one?

```
currElement = currElement + 1;
```

while(){…}

- While not at the end of the list, print out the next element

```
int currElement = 0;
while(currElement != list.length)
{
      System.out.println(list[currElement]);
      currElement = currElement + 1;
}//end loop while(currElement!=list.length)
```

Time to Optimize

- `currElement = currElement + 1;`
- This is really long. There must be a faster way
- There is! Use +=

+=

- Adds the right side to whatever was on the left side

```
currElement += 1;
```

is equivalent to

```
currElement = currElement + 1;
```

- As it happens, people generally add one to many things all the time, so there is a shorter-cut

++

```
currElement++;
```

is equivalent to

```
currElement = currElement + 1;
```

- It is also common to subtract one from many things all the time, so there is a similar shorter-cut

--

```
currElement--;
```

is equivalent to

```
currElement = currElement - 1;
```

- Planning to add or subtract more than one? Use += and -=

Mathematical Combination Operators

`+=, -=, *=, /=` (any numeric primitive)

`%=` (`int` only)

- Literally combines the operation and the assignment.

variable ◊= value variable = variable ◊ value;

```
int x = 5;
x %= 3; //x == ?
```

while(){…}

```
int currElement = 0;
while(currElement != list.length)
{

  System.out.println(list[currElement]);
      currElement++;
}//end loop while(currElement!
  =list.length)
```

A Little Security

- `currElement != list.length`
- What happens if you accidentally added by three instead of one?
- What happens if you accidentally changed the value of **`currElement`** so it's past the list length?
- Safer check: `currElement < list.length`

while(){…}

```
int currElement = 0;
while(currElement < list.length)
{

  System.out.println(list[currElement]);
      currElement++;
}//end loop
  while(currElement<list.length)
```

So Many Things We Have to Remember...WHYYYYYYYYYY

```
int counter = start;
while(condition)
{
        //Do stuff
        counter increment
}//end loop while(condition)
```

- There must be a better way
- WILD KEYWORD APPEARS

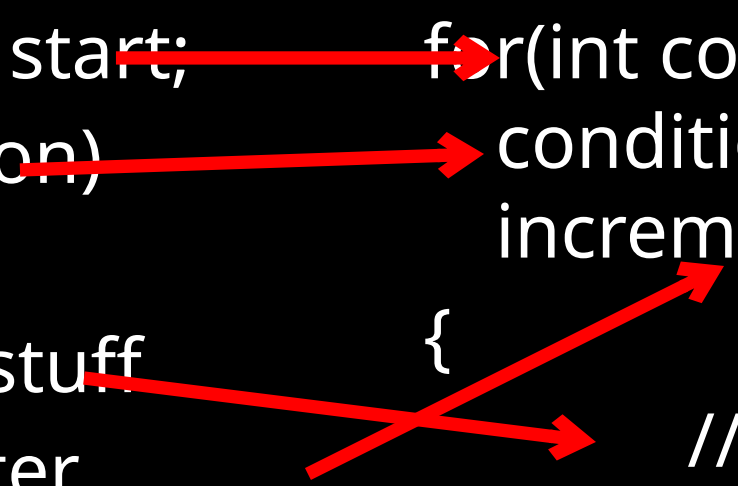
`for(;;){…}`

- The for loop allows us to put everything together for us!
- for(counter = start; condition; counter increment)
- Now we don't have to worry about the placement of the counter and incrementing it!

EQUIVALENCE

```
int counter = start;
while(condition)
{
        //Do stuff
        counter
   increment
}//end loop
  //while(condition)
```

```
for(int counter = start;
   condition; counter
   increment)
{
        //Do stuff
}//end loop
   for(condition)
```



for(;;){…}

```
for(int currElement = 0; currElement <
  list.length; currElement++)
{

  System.out.println(list[currElement]);
}//end loop for(currElement <
  list.length)
```